

Project #1

OpenMP : Monte Carlo Simulation

Woonki Kim

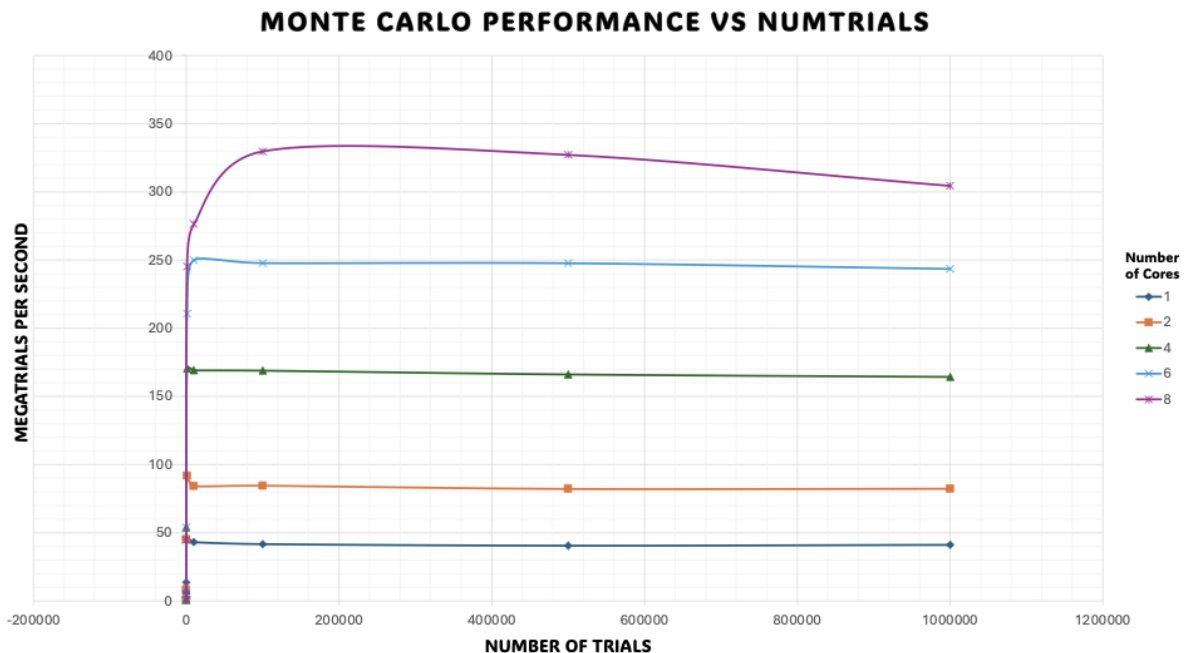
kimwoon@oregonstate.edu

14 April 2024

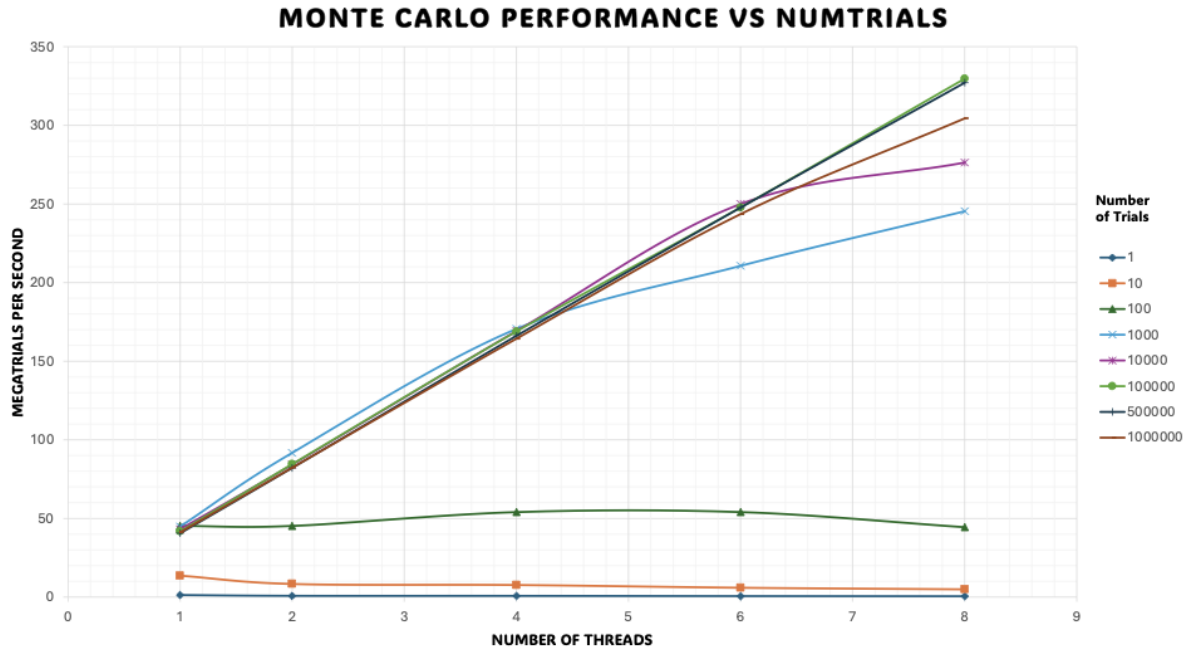
1. Table of the performance.

Number of Cores/Number of Trials	1	10	100	1000	10000	100000	500000	1000000
1	1.29	13.59	45.23	44.83	43.1	41.68	40.53	41.12
2	0.77	8.31	45.23	91.71	84.25	84.5	82.14	82.25
4	0.73	7.65	54	170.5	169.13	168.88	166.1	164.28
6	0.59	5.9	54.02	210.61	249.94	247.77	247.66	243.51
8	0.5	4.89	44.41	245.34	276.32	329.65	327.08	304.41

2. Performance versus the number of Monte Carlo trials.



3. Performance versus the number OpenMP threads.



4. Estimation of the probability.

- Here is table with column as Number of Trials, and row as Number of Cores, and the values as probabilities.

Number of Cores/Number of Trials	1	10	100	1000	10000	100000	500000	1000000
1	100	50	51	58	57.51	57.24	57.09	57.08
2	0	40	53	59.8	57.06	57.07	56.96	56.94
4	100	60	63	57.3	57.24	57.02	56.98	56.97
6	0	30	57	56.8	57.55	57.13	57.05	57
8	100	40	63	57.4	57.12	57	56.89	56.98

- You can see the probability range narrows down as number of trials increases. It is obvious that probability calculated with higher trials would be more accurate. So if you see the probabilities with 1,000,000 trials, all of them are close to 57%.

5. Estimation of the Parallel Fraction.

- The Parallel Fraction is :

$$F_p = \frac{n}{n-1} \left(1 - \frac{1}{\text{Speedup}} \right) = \frac{n}{n-1} \left(1 - \frac{\text{MegaTrialsPerSec}_1}{\text{MegaTrialsPerSec}_n} \right)$$

- To get a Parallel Fraction for thread 2 for 1,000,000 trials.

$$F_p = \frac{2}{1} \left(1 - \frac{1}{\text{Speedup}} \right) = \frac{2}{1} \left(1 - \frac{\text{MegaTrialsPerSec}_1}{\text{MegaTrialsPerSec}_2} \right) = \frac{2}{1} \left(1 - \frac{41.12}{82.25} \right)$$

$$\approx \frac{2}{1} (1 - 0.5001) = \frac{2}{1} (0.4998) \approx 0.999$$

- To get a Parallel Fraction for thread 4 for 1,000,000 trials.

$$F_p = \frac{4}{3} \left(1 - \frac{1}{\text{Speedup}} \right) = \frac{4}{3} \left(1 - \frac{\text{MegaTrialsPerSec}_1}{\text{MegaTrialsPerSec}_4} \right) = \frac{4}{3} \left(1 - \frac{41.12}{164.28} \right)$$

$$\approx \frac{4}{3} (1 - 0.2503) = \frac{4}{3} (0.7496) \approx 0.999$$

- To get a Parallel Fraction for thread 6 for 1,000,000 trials.

$$F_p = \frac{6}{5} \left(1 - \frac{1}{\text{Speedup}} \right) = \frac{6}{5} \left(1 - \frac{\text{MegaTrialsPerSec}_1}{\text{MegaTrialsPerSec}_6} \right) = \frac{6}{5} \left(1 - \frac{41.12}{243.51} \right)$$

$$\approx \frac{6}{5} (1 - 0.1688) = \frac{6}{5} (0.8311) \approx 0.997$$

- To get a Parallel Fraction for thread 8 for 1,000,000 trials.

$$F_p = \frac{8}{7} \left(1 - \frac{1}{\text{Speedup}} \right) = \frac{8}{7} \left(1 - \frac{\text{MegaTrialsPerSec}_1}{\text{MegaTrialsPerSec}_8} \right) = \frac{8}{7} \left(1 - \frac{41.12}{304.41} \right)$$

$$\approx \frac{8}{7} (1 - 0.1350) = \frac{8}{7} (0.8311) \approx 0.9885$$

- The consistency of parallel fractions across different thread numbers suggests that the parallel portion of the process remains consistently low, indicating a greater proportion of sequential processing.

6. Analysis of the Graphs.

- Monte Carlo Performance VS NumTrials Graph
 - The graph shows that employing multiple threads provides an advantage for faster calculations. MegaTrialsPerSecond remains constant for thread counts of 1, 2, 4, and 6, whereas there's a slight decrease for 8 threads. This decrease suggests that the calculation with 8 threads may incur additional context switching time, leading to a slightly slower

performance. While using a higher number of threads generally improves performance, it's not always the most efficient approach.

- Performance versus the number OpenMP threads Graph.
 - After conducting 1-to-1,000,000 trials, it's evident from the graph that the number of trials remains constant at under 100 regardless of the number of threads used. This suggests that employing only one thread is sufficient for achieving maximum performance in calculation for under 100 trials.
 - The graph demonstrates that for trials exceeding 100, the MegaTrialsPerSecond increases proportionally with the number of threads used. This outcome highlights the potential for dynamic calculation enhancement through the utilization of multiple threads.
 - The graph also reveals that the number of trials at 500,000 and 1,000,000 are almost identical. This implies that the optimal point for maximizing the benefits of increasing threads is reached at around 500,000 trials, particularly when utilizing 8 threads. Employing 10 threads will likely yield a noticeable difference between 500,000 and 1,000,000 trials.